My part of the projects:

Project 1:

```
void Withdraw(int id) {
                    float w;
                    std::cout << "Please enter amount of money you wish to withdraw ";
                    std::cin >> w;
                    int n = Trans[id].size();
                    n--;
                    float temp = Trans[id][n];
                    temp = temp - w;
                    Trans[id].push_back(temp);
                    InMess[id].push_back(to_string(w) + " w");
}
void Deposit(int id) {
          float d;
          std::cout << "Please enter amount of money you wish to deposit ";
          std::cin >> d;
          int n = Trans[id].size();
          n--;
          float temp = Trans[id][n];
          temp = temp + d;
          Trans[id].push_back(temp);
          InMess[id].push_back(to_string(d) + " D");
}
int TranID(int id) {
          int newid;
          std::cout << "please enter the new id you want between 0-19 ";
          std::cin >> newid;
          while (CusInfo[newid][0] != "") {
                    std::cout << "please enter different id the one you want is already taken between 0-19 ";
                    std::cin >> newid;
          }
          //put all the values in the new place
          for (int i = 0; i < 10; i++) {
                    CusInfo[newid][i] = CusInfo[id][i];
          }
          //empty array
          for (int i = 0; i < 10; i++) {
                    CusInfo[id][i] = "";
          }
          int n = Trans[id].size();
          //put all the values in the new place
          for (int i = 0; i < n; i++) {
                    Trans[newid].push_back(Trans[id][i]);
          }
          //empty vector
          Trans[id].clear();

          int k = InMess[id].size();
          //put all the values in the new place
          for (int i = 0; i < k; i++) {
                    InMess[newid].push_back(InMess[id][i]);
          }
          //empty vector
          InMess[id].clear();
          return 7;

}
void LastTran(int id,int N) {
          int n = Trans[id].size();

          if (N < n) {
                    n = n - 1;
                    float tran;
                    std::cout << "Last " << N << " Transactions:" << endl;
                    int temp = Trans[id][n];
                    for (int i = n - 1; i >= n - N; i--) {
                              tran = temp - Trans[id][i];
                              temp = Trans[id][i];
                              std::cout << tran << endl;
                    }
          }
          else {

                    cout << "too many transaction you only had: " << n-1 << " transactions" << endl;
          }

}
void Currentbal(int id) {
          int n = Trans[id].size();
          n = n - 1;
          std::cout <<"your balance is: "<< Trans[id][n]<<endl;
}
void CheckInbox(int id) {
          int n = InMess[id].size();
          for (int i = 0; i < n; i++) {
                    std::cout << InMess[id][i] << endl;
```

```
		}
}
void CustomerFun(int id) {
		int k = 0;
		if (CusInfo[id][5] == "Active") {
			while (k != 7) {
				std::cout << " ************WelcometoC++ProgrammersBank********************" << endl;
				std::cout << "1. Statement summary last N transaction: " << endl;
				std::cout << "2. Current Balance: " << endl;
				std::cout << "3. Withdraw: " << endl;
				std::cout << "4. Deposit:" << endl;
				std::cout << "5. Tranfer to other CID: " << endl;
				std::cout << "6. Check Inbox: " << endl;
				std::cout << "7. logout: " << endl;
				std::cin >> k;

				if (k == 1) {
					int n;
					std::cout << "enter your desire last transactions: ";
					std::cin >> n;
					LastTran(id, n);
				}
				if (k == 2) {
					Currentbal(id);
				}
				if (k == 3) {
					Withdraw(id);
				}
				if (k == 4) {
					Deposit(id);
				}
				if (k == 5) {
					k = TranID(id);
				}
				if (k == 6) {
					CheckInbox(id);
				}
			}
		}
		else {
			std::cout << "This account in blocked";
		}
}
int main() {
		int i = 0;
		while (i!=4) {
			std::cout <<  " ************WelcometoC++ProgrammersBank********************"<<endl;
			std::cout << "1. Admin sign in"<<endl;
			std::cout << "2. Costumer sign in page"<<endl;
			std::cout << "3. CustomerSignUppage:"<<endl;
			std::cout << "4. Exit application"<<endl;
			std::cin >> i;
			if (i == 1) {
				std::string user, pass;
				std::cout << "Enter user name ";
				cin >> user;
				std::cout << "Enter Admin password name ";
				cin >> pass;
				if (user == Adminuser && pass == AdminPass) {
					AdminFun();
				}
				else
				{
					cout << "Wrong password or username " << endl;
				}

			}
			if (i == 3) {
				CustomerSignUp();

				i = 2;
			}
			while(i == 2) {
				std::string user, pass;
				std::cout << "Enter user name ";
				cin >> user;
				while(!checkUserName(user)&&i!=3) {
					std::cout << "User name was not found please enter a different one or back for sign up page
";

					cin >> user;
					if (user == "back") {
						i = 3;
					}
				}
				if (i == 3) {
					break;
				}
				std::cout << "Enter password ";
				cin >> pass;
				if (checkPassword(user, pass)) {
					CustomerFun(findCID(user));
					i = 0;
```

```
                                }
                                else {
                                        std::cout << "Password and user name does not match " << endl;;
                                        i = 2;
                                }
                        }
                }


}

int findCID(std::string user) {
        int id = -1;
        for (int i = 0; i < 20; i++) {
                if (CusInfo[i][1] == user) {
                        id = i;
                }
        }
        return id;
}
```

Project 2:

```
int main() {
        int i = 0;
        admin r = CreateAdmin();
        Cus = Readtextfile(Cus);
        Cus = ReadTransaction(Cus);
        Cus = ReadMessages(Cus);
        while (i != 4) {
                std::cout << " ************WelcometoC++ProgrammersBank********************" << endl;
                std::cout << "1. Admin sign in" << endl;
                std::cout << "2. Costumer sign in page" << endl;
                std::cout << "3. CustomerSignUppage:" << endl;
                std::cout << "4. Exit application" << endl;
                std::cin >> i;
                if(i==1){
                        string pass, user;
                        cout << "Enter User name: ";
                        cin >> user;
                        cout << "Enter password: ";
                        cin >> pass;
                        if (r.getPass() == pass && r.getUsername() == user) {
                                AdminFun(r);
                        }
                        else {
                                cout << "Wrong Password";
                        }
                }
                else if (i==2){
                        Cuslogin(Cus);
                }
                else if (i==3){
                        CustomerSignUp();
                }

        }
        WriteCus(Cus);
        WriteTransaction(Cus);
        WriteMessages(Cus);
}

void CustomerFun(customer c) {
        int k = 0;
        if (c.GetStatus() == "Active") {
                while (k != 7) {
                        std::cout << " ************WelcometoC++ProgrammersBank********************" << endl;
                        std::cout << "1. Statement summary last N transaction: " << endl;
                        std::cout << "2. Current Balance: " << endl;
                        std::cout << "3. Withdraw: " << endl;
                        std::cout << "4. Deposit:" << endl;
                        std::cout << "5. Tranfer to other CID: " << endl;
                        std::cout << "6. Check Inbox: " << endl;
                        std::cout << "7. logout: " << endl;
                        std::cin >> k;

                        if (k == 1) {
                                int n;
                                std::cout << "enter your desire last transactions: ";
                                std::cin >> n;
                                c.LastTran(n);
                        }
                        if (k == 2) {
                                c.curBalance();
                        }
                        if (k == 3) {
                                float n;
                                std::cout << "enter your desire withdrawl: ";
```

```
                                        std::cin >> n;
                                        c.withdraw(n);
                                }
                                if (k == 4) {
                                        float n;
                                        std::cout << "enter your desire withdrawl: ";
                                        std::cin >> n;
                                        c.deposit(n);
                                }
                                if (k == 5) {
                                        int n;
                                        std::cout << "enter your desire new id: ";
                                        std::cin >> n;
                                        while (!CheckId(Cus, n)) {
                                                std::cout << "This ID is taken please choose another one: ";
                                                std::cin >> n;
                                        }
                                        c.SetId(n);
                                }
                                if (k == 6) {
                                        c.ShowMessages();
                                }
                        }
                }
                else {
                        std::cout << "This account in blocked";
                }
}

vector<customer> ReadMessages(vector<customer> c) {
                if (!c.empty()) {
                        ifstream inputfile;
                        string line;
                        int j = 0;
                        inputfile.open("Messages.txt");
                        if (!inputfile.is_open()) {
                                cout << "File is not opened";
                        }
                        else {
                                while (getline(inputfile, line)) {
                                        string str = "";
                                        int n = line.length();
                                        for (int i = 0; i <= n; i++) {


                                                if (line[i] == ' ' || i == (n - 1)) {
                                                        c[j].GetMess().push_back(str + line[i]);
                                                        str = "";
                                                }
                                                else {
                                                        str = str + line[i];
                                                }
                                        }
                                        j++;

                                }
                        }
                        inputfile.close();

                }
                return c;
}
void WriteTransaction(vector<customer> c) {
                if (!c.empty()) {
                        int n = c.size();
                        ofstream file1;
                        file1.open("Transaction.txt");
                        if (!file1.is_open()) {
                                cout << "File is not opened";
                        }
                        for (int i = 0; i < n; i++) {
                                int x = c[i].GetTran().size();
                                for (int j = 0; j < x; j++) {
                                        file1 << c[i].GetTran()[j] << " ";
                                }
                                file1 << "\n";
                        }
                        file1.close();

                }
}
void WriteMessages(vector<customer> c) {
                if (!c.empty()) {
                        int n = c.size();
                        ofstream file1;
                        file1.open("Messages.txt");
                        if (!file1.is_open()) {
                                cout << "File is not opened";
                        }
                        for (int i = 0; i < n; i++) {
                                int x = c[i].GetMess().size();
```

```
                                vector<string> mes = c[i].GetMess();

                                for (int j = 0; j < x; j++) {
                                        file1 << mes[j] << " ";
                                }
                                file1 << "\n";
                        }
                        file1.close();
                }
}
int CheckId(vector<customer> c) {
                int k = 0;
                if (c.empty()) {
                        return k;
                }
                else {
                        int n = c.size();


                        for (int j = 0; j < n; j++) {
                                for (int i = 0; i < n; i++) {
                                        if (c[i].GetID() == k) {
                                                k++;
                                        }
                                }
                                if (k == c.size() - 1) {
                                        break;
                                }


                        }
                        return k;
                }
}

admin CreateAdmin() {
                admin r = admin();
                r.setPass("1234");
                r.setUsername("galio");
                r.setID(1);
                r.setSSN("12345");
                r.setName("Ido");
                r.setOrganization("UC");
                r.setDOB("07/12/2000");
                r.setDOJ("y");
                r.SetRank("Manager");
                return r;
}


string Incrypt(string str) {
                int n = str.length();
                char temp;
                string newstr;
                if (n % 2 == 0) {
                        for (int i = 0; i < n; i++) {
                                temp = str[i] + 3;
                                newstr += temp;
                        }
                }
                else {
                        for (int i = 0; i < n; i++) {
                                temp = str[i] + 4;
                                newstr += temp;
                        }
                }
                return newstr;
}

string Decrypt(string str) {
                int n = str.length();
                char temp;
                string newstr;
                if (n % 2 == 0) {
                        for (int i = 0; i < n; i++) {
                                temp = str[i] - 3;
                                newstr += temp;
                        }
                }
                else {
                        for (int i = 0; i < n; i++) {
                                temp = str[i] - 4;
                                newstr += temp;
                        }
                }
                return newstr;
}
string get_hash(string str) {
                int n = str.length();
                const int p = 31, m = 1e9 + 7;
                long long hash = 0;
                long p_pow = 1;
```

```
	for (int i = 0; i < n; i++) {
		if (str[i] > 96) {
			hash = ((hash + str[i] - 'A' + 1) * p_pow) % m;
			p_pow = (p_pow * p) % m;
		}


	}
	return to_string(hash);
}

void WriteCus(vector<customer> c) {
	if (!c.empty()) {
		int n = c.size();
		ofstream file1;
		file1.open("CusInfo.txt");
		if (!file1.is_open()) {
			cout << "File is not opened";
		}
		for (int i = 0; i < n; i++) {
			string str = to_string(c[i].GetID());
			string user, pass, name, type, Org, Dob, SSN, Doj,warning;
			user = c[i].GetUSER();
			pass = c[i].GetPassword();
			name = c[i].GetNAME();
			type = c[i].GetType();
			Org = c[i].GetORG();
			Dob = c[i].GetDob();
			SSN = c[i].GetSsn();
			Doj = c[i].GetDoj();
			warning = to_string(c[i].GetWarning());
			int balance = c[i].GetBalance();
			file1 << (str) << " " << Incrypt(user) << " " << pass << " " << Incrypt(name) << " " << Incrypt(type)
<< " " << Incrypt(Org) << " " << Incrypt(Dob) << " " << Incrypt(SSN) << " " << Incrypt(Doj) <<" "<< warning <<" "<< balance<< endl;
		}
		file1.close();
	}
}

vector<customer> ReadTransaction(vector<customer> c) {
	if (!c.empty()) {
		ifstream inputfile;
		string line;
		int j = 0;
		inputfile.open("Transaction.txt");
		if (!inputfile.is_open()) {
			cout << "File is not opened";
		}
		else {
			while (getline(inputfile, line)) {
				string str = "";
				int n = line.length();
				for (int i = 0; i <= n; i++) {


					if (line[i] == ' ' || i == (n - 1)) {
						c[j].GetTran().push_back(stof(str + line[i]));
						str = "";
					}
					else {
						str = str + line[i];
					}
				}
				j++;

			}
		}
		inputfile.close();
	}
	return c;
}

void deposit(float num) {
		SetBalance(GetBalance()+num);
		int n = transactions.size();
		if (!transactions.empty()) {
			n = n - 1;
			transactions.push_back(transactions[n] + num);
		}
		else {
			transactions.push_back(num);
		}
		messages.push_back(to_string(num) + "D");
	}
	void withdraw(float num) {
		SetBalance(GetBalance() - num);
		int n = transactions.size();
		transactions.push_back(transactions[n - 1] - num);
		messages.push_back(to_string(num) + "W");
	}
	void ShowMessages() {
		int n = messages.size();
```

```
        for (int i = 0; i < n; i++) {
                cout << messages[i] << endl;
        }
}
void curBalance() {
        cout << "Balance: " << balance << endl;
}
void LastTran(int N) {
        int n = transactions.size();

        if (N < n) {
                n = n - 1;
                float tran;
                std::cout << "Last " << N << " Transactions:" << endl;
                int temp = transactions[n];
                for (int i = n - 1; i >= n - N; i--) {
                        tran = temp - transactions[i];
                        temp = transactions[i];
                        std::cout << tran << endl;
                }
        }
        else {

                cout << "too many transaction you only had: " << n - 1 << " transactions" << endl;
        }
}
```